

Short recap day #1



Peter Kassenaar – info@kassenaar.com

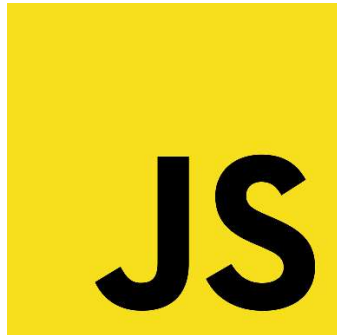
Front-end : **one** stack

- HTML, CSS, JavaScript
- Huge landscape – lots modules in one stack

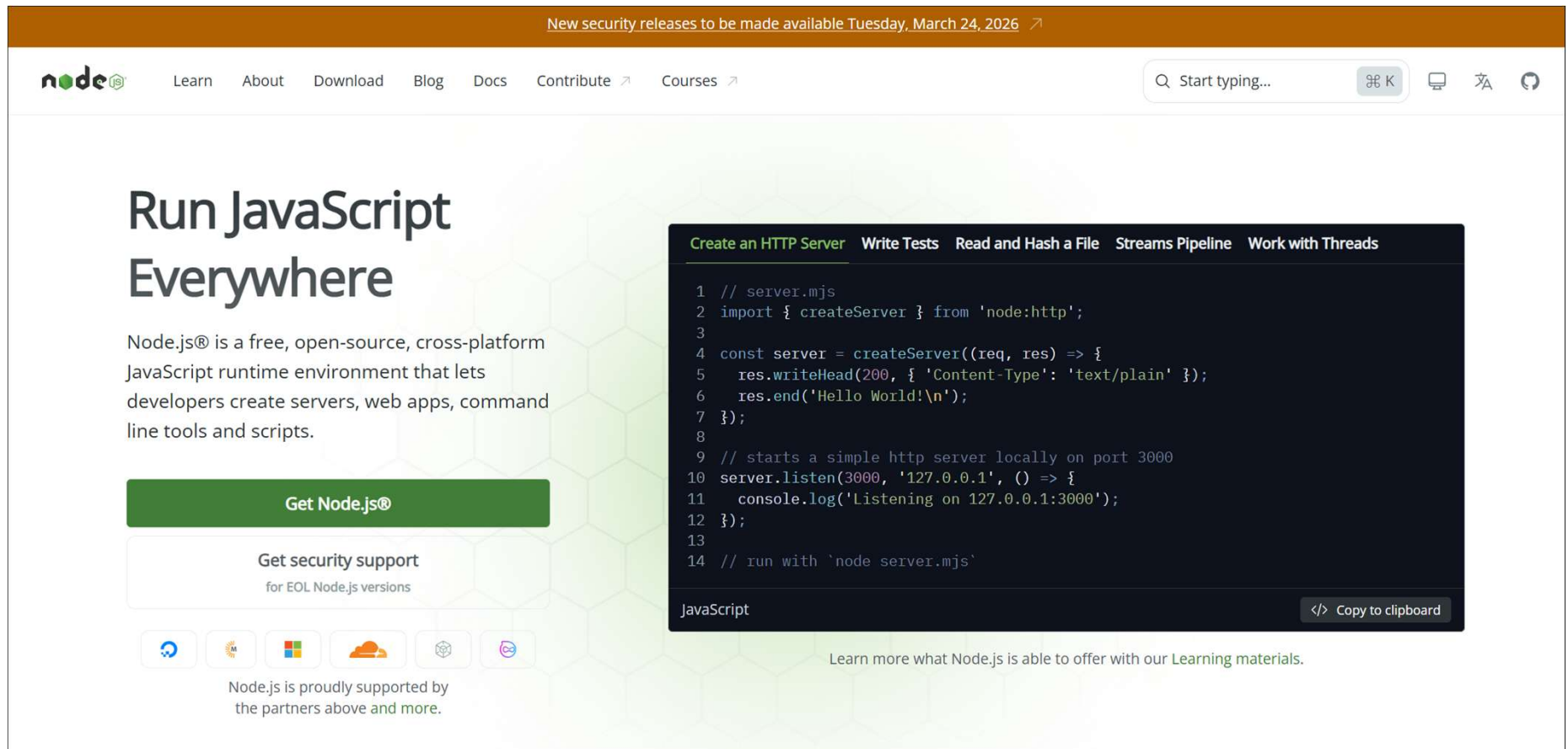
HTML



CSS



Managed in one central tool



The screenshot shows the Node.js website homepage. At the top, there is a navigation bar with links: Learn, About, Download, Blog, Docs, Contribute, and Courses. A search bar is on the right. The main heading is "Run JavaScript Everywhere". Below it, a paragraph describes Node.js as a free, open-source, cross-platform JavaScript runtime environment. A green button labeled "Get Node.js®" is prominent. Below this, there is a section for "Get security support for EOL Node.js versions" and a row of partner logos (Docker, AWS, Microsoft, etc.). A code editor window is overlaid on the right, showing a JavaScript code snippet for creating an HTTP server. The code includes comments and uses the `createServer` function from `node:http`. A "Copy to clipboard" button is at the bottom right of the code editor. At the bottom of the page, there is a link to "Learn more what Node.js is able to offer with our Learning materials."

New security releases to be made available Tuesday, March 24, 2026 ↗

node.js

Learn About Download Blog Docs Contribute ↗ Courses ↗

Q Start typing... ⌘ K

Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

Get Node.js®

Get security support
for EOL Node.js versions

Node.js is proudly supported by the partners above and more.

Create an HTTP Server Write Tests Read and Hash a File Streams Pipeline Work with Threads

```
1 // server.mjs
2 import { createServer } from 'node:http';
3
4 const server = createServer((req, res) => {
5   res.writeHead(200, { 'Content-Type': 'text/plain' });
6   res.end('Hello World!\n');
7 });
8
9 // starts a simple http server locally on port 3000
10 server.listen(3000, '127.0.0.1', () => {
11   console.log('Listening on 127.0.0.1:3000');
12 });
13
14 // run with `node server.mjs`
```

JavaScript </> Copy to clipboard

Learn more what Node.js is able to offer with our [Learning materials](#).

<https://nodejs.org/en>

Alternatives to NodeJS

Bun is joining Anthropic & Anthropic

 **Bun**

Bun v1.3.11 is here! →

Bun is a *fast* JavaScript all-in-one toolkit

Bun is a fast, **incrementally adoptable** all-in-one JavaScript runtime, bundler, test runner, and package manager built in. Bun is Node.js compatible.

 **deno** Products ▾ Docs Modules ▾ Community ▾ Blog

Search 36K

Uncomplicate JavaScript

Deno is the open-source JavaScript runtime for the modern web.

[Docs >](#) [GitHub >](#)



facebook / hermes

Code Issues 125 Pull requests 66 Discussions Actions Projects Models Security Insights

Files

- lib
- lldb
- npm
- public
- test
- tools
- unittests
- unsupported
- utils
- .clang-format
- .clang-tidy

hermes / README.md

ykws and facebook-github-bot Fix README Logo Image (#1373) ce90d24 · 2 years ago

Preview Code Blame 61 lines (41 loc) · 3.54 KB

Hermes JS Engine

license MIT npm v0.11.0 oss-fuzz build failing PRs welcome

Hermes is a JavaScript engine optimized for fast start-up of [React Native](#) apps. It features ahead-of-time static optimization and compact bytecode.

If you're only interested in using pre-built Hermes in a new or existing React Native app, you do not need to follow this guide or have direct access to the Hermes source. Instead, just follow [these instructions to enable Hermes](#).

Noted that each Hermes release is aimed at a specific RN version. The rule of thumb is to always follow [Hermes releases](#) strictly.



QUASAR

BEYOND THE FRAMEWORK

Ready cross-platform VueJs frame



*Bottomline – you **don't have to learn** them, but be aware of their existence. There are node.js **alternatives with the same goal***

NodeJS : 2 types of Modules

- CommonJS Modules:

- `myModule.js` → `Module.exports = myFunction` // export
- `const myFunction = require('./myModule')` // import
- Still used A LOT in existing code bases

- ES Modules (modern, but not widespread yet)

- `myModule.js` → `export default myFunction = "..."` // export
- `import myFunction from './myModule'` // import
- Better treeshaking, more modern notation
- But: NO NEED to refactor/switch module systems. Node works perfectly fine with both



+



Express.js framework

express

search

Getting started

Guide

API reference

Advanced topics

Resources

Support

Blog

🌙

🌐

Express 5.2.1

Fast, unopinionated, minimalist web framework for [Node.js](#)

```
$ npm install express --save
```

```
const express = require('express')
const app = express()
const port = 3000

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`)
})
```

 **Express@5.1.0: Now the Default on npm with LTS Timeline**

Express 5.1.0 is now the default on npm, and we're introducing an official LTS schedule for the v4 and v5 release lines. [Check out our latest blog for more information.](#)

Web Applications

Express is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.

APIs

With a myriad of HTTP utility methods and middleware at your disposal, creating a robust API is quick and easy.

Performance

Express provides a thin layer of fundamental web application features, without obscuring Node.js features that you know and love.

Middleware

Express is a lightweight and flexible routing framework with minimal core features meant to be augmented through the use of Express [middleware](#) modules.

 **OpenJS**
Foundation

<https://expressjs.com/>

Node.js – Peter Kassenaar

Express: twee manieren

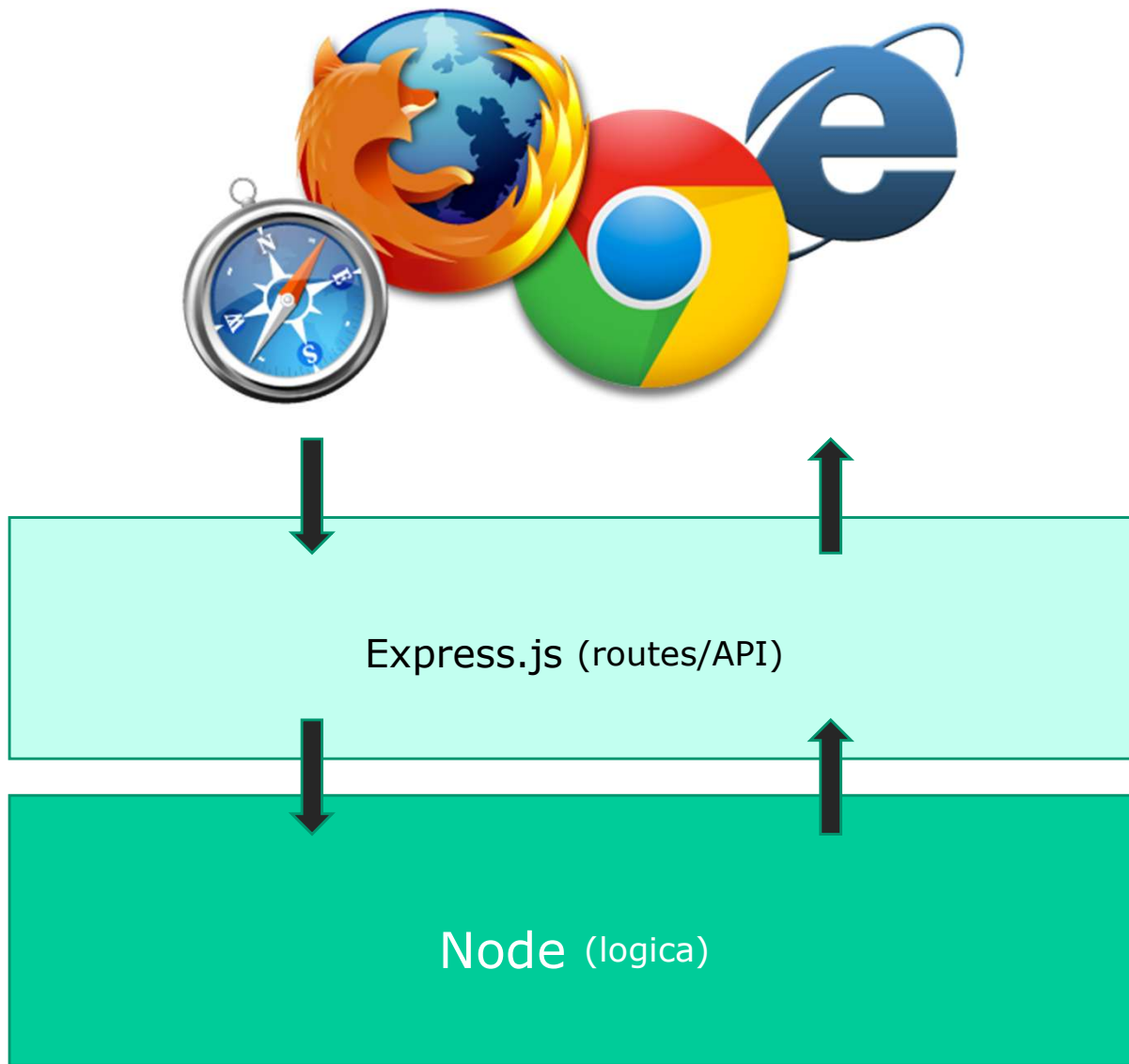
1. Globally: `npm install -g express-generator`

- Generator
- Snelle scaffolding van kale basiswebsite
- View Engines (Jade, Pug, EJS)
- Voordeel: snel op weg
- Nadeel: veel code die je wellicht niet wilt gebruiken, of volgens andere conventies

2. Locally: `npm install express`, per project

- Creating API's

Middleware



Express API maken

Routes schrijven binnen je app

```
app.get('/api/auteurs', function (req, res) {  
  // retourneer gegevens, hier een JSON-bestand met auteurs  
  res.json(auteurs);  
});
```

- Elke functionaliteit krijgt een eigen RESTful route
- 20 routes? → 20 verschillende endpoints
- Routeparameters: dubbele punt / dynamische routes

```
app.get('/api/boeken/:id', function (req, res) {...});
```


Questions: Node Performance?

- Overall impressions, common ideas:
- Rust is generally the fastest
 - Rust handles 72.000 req/s vs Node.js 8.000 – 15.000 req/sec, depending on RAM and tasks
- Node.js sits in the middle
 - much faster than Python for I/O-heavy work, slower than Rust/Go for CPU-intensive tasks.
- Python is the slowest for raw computation
 - libraries like NumPy/PyTorch offload to native code, so in practice it's more nuanced.

Some benchmarks:

- <https://programming-language-benchmarks.vercel.app/rust-vs-javascript>
- <https://www.techempower.com/benchmarks/#section=data-r23>
- <https://www.wwt.com/blog/performance-benchmarking-bun-vs-c-vs-go-vs-nodejs-vs-python>

[Blog](#) [Python](#) [Go](#) [Node.js](#) [.NET Core](#) [+4](#)

Blog • June 13, 2023 • 6 minute read

Performance Benchmarking: Bun vs. C# vs. Go vs. Node.js vs. Python

I took 5 runtimes and built functionally identical REST APIs with each of them. Then I ran load tests for each application to measure their performance.



Every once in a while, it can be useful to benchmark several different languages or runtimes to see which perform the best. That's simply because things are always changing. Languages tend to get better over time and new alternatives are always just around the corner.

In addition to that, I have an interest in performance that stems from my work with OpenGL more than 20 years ago. I used to obsess over frame rates of my 3D applications. Back then I

[in](#) [f](#) [X](#) [✉](#)

[7](#) [1](#) [🔖](#)

Contributors

AI on benchmarks:

"Node.js is V8 JIT-compiled so it's surprisingly fast for dynamic code, but it's single-threaded by nature (event loop).

Rust has no GC and compiles to machine code, so it wins on CPU and memory.

Python's GIL is its big bottleneck for parallelism.

The real-world answer always depends on the workload type"

Using multiple import types in a module?

- Not recommended
- Possible in **one direction**: Importing CommonJS in ESM, NOT the other way around.
- See example `300-mixed-imports`

```
// Modern ESM imports:
import { createRequire } from 'module';
import { add } from './math.mjs';

// Bridge to load old-skool CJS modules:
const require: Require = createRequire(import.meta.url); // edito
const { greet } = require('./greeter.cjs');

console.log(greet('Peter')); // Hello, Peter!
console.log(add(2, 3));      // 5
```


Today:

- Finish Express – API's and middleware
- React:
 - Concepts
 - Structure + architecture + building
 - Components (classes + functions)
 - Hooks
 - State
 - Props
 - Data

Tomorrow

- React Data (cont'd)
 - Displaying details
- React Forms
 - user input
- External data / API's
- Evals/goodbye